Compatibility Findings: Classical Mechanics and FakeScience

Formalization notes

November 13, 2026

1 Findings

The two supplied Lean files are not jointly importable as written. Both declare the fully qualified name `ClassicalMechanics.secondLawHolds`, but give it different types. In `ClassicalMechanics.lean`, it states, for a system M and a particle p at time t,

$$\text{forceSum}(M.\text{force}(p,t)) = \text{scale}(M.\text{mass}(p), M.\text{acceleration}(p,t)).$$

This extracts Newton's second law from the system's assumptions; it is the P_6 clause of the axiomatic presentation [1]. In `FakeScience.lean`, the same name is instead a theorem of type `False`. Lean rejects both a joint import and the combined declarations because the name is already declared. This diagnostic is a *name collision*, not a theorem that the two physical claims contradict each other.

The fake file also declares

```
axiom ClassicalMechanics.physics :  False.
```

This is a different and deeper problem: the file assumes a proof of `False`. From that assumption Lean's `False.elim` derives any proposition Q, so the resulting theory is logically inconsistent relative to ordinary Lean logic. Lean accepts the file because it permits users to introduce axioms; acceptance does not validate those axioms. The fake theorem explicitly depends on `physics`; the genuine `secondLawHolds` has no declared axiom dependencies. The supplied fake file does not state an alternative force law or supply a model, so no specific physical conflict with P_6 can be inferred from it. Renaming the duplicate theorem would resolve the name collision, but would not repair the false axiom.

2 Reproduction

With Lean 4 installed, run from the repository root:

```
bash _lean/check_compatibility.sh
```

The script compiles both files separately, checks the axiom dependencies, and verifies that joint import and combined declarations fail with the expected name-collision diagnostics. It also checks `FakeScienceAudit.lean`, which derives an arbitrary proposition from the fake axiom and reports the dependency using `#print axioms`. The checks reported here were run with Lean 4.35.0; the script checks specific diagnostic wording, which may change between Lean versions. On a machine using `elan` without a default toolchain, set `ELAN_TOOLCHAIN` to an installed Lean 4 toolchain first.

These checks do not prove that the abstract classical mechanics signature has a concrete real-analysis interpretation, nor do they check the truth of any empirical paper. They locate the collision and show exactly how the fake file's explicit assumption makes arbitrary propositions derivable.

3 Predicate-Based Formulation: Second Example

In addition to the structure-based approach above, a second example demonstrates the explicit predicate formulation:

$$\mathrm{CM}(\mathcal{M}) \quad \Longleftrightarrow \quad P_1 \wedge P_2 \wedge \cdots \wedge P_6.$$

This predicate-based approach uses a direct logical definition rather than a bundled type structure.

3.1 Predicate Definition

The file `ClassicalMechanicsPredicate.lean` defines

`isClassicalMechanicsModel` : Prop,

which explicitly conjoins all six axiom conditions. The theorem `cm_iff_conjunction` establishes the biconditional:

$$\texttt{isClassicalMechanicsModel}(S, M) \quad \Longleftrightarrow \quad P_1(S,M) \wedge P_2(S,M) \wedge \cdots \wedge P_6(S,M).$$

This makes the transparent, conjunctive structure of model membership explicit.

3.2 Incompatibility in Predicate Form

The file `FakeSciencePredicate.lean` redefines `isClassicalMechanicsModel` in the same namespace, introducing the false axiom

```
axiom ClassicalMechanicsPredicate.conflict :  False.
```

This creates the same two failure modes as Example 1:

1. **Redeclaration conflict**: Lean rejects both joint import and combined declarations because `isClassicalMechanicsModel` cannot be redefined in the same namespace.

2. **False axiom**: Any proposition derived from `FakeSciencePredicate` is falsely proven, violating consistency.

3.3 Audit and Detection

The file `PredicateAudit.lean` attempts to combine both modules, which triggers Lean's incompatibility detection. The theorem demonstrates the axiom dependency:

```
theorem predicate_depends_on_false :
exists p :  Prop, p = FakeSciencePredicate.conflict
```

This establishes that when both modules are present, the Lean type checker reports the redeclaration error and halts compilation, proving that the two formulations cannot coexist. The proof witnesses the false axiom `FakeSciencePredicate.conflict`.

4 Comparison: Structure vs. Predicate Approaches

Both approaches are equivalent in logical power and demonstrate that incompatibility between rival theories is decidable and machine-checkable. The predicate-based formulation emphasizes the logical transparency of the Suppes tradition, making the conjunction of axioms and the biconditional definition explicit in the source code.

5 Running Both Examples

The bash script `check_compatibility.sh` now includes both examples:

Aspect	Structure-Based (Example 1)	Predicate-Based (Example 2)
Model Encoding	Bundled `System` structure	Explicit test function
Axiom Encoding	Constructor fields with proofs	Logical conjunction
Selection	Type instantiation	Predicate evaluation
Transparency	Conditions in type structure	Biconditional makes equivalence manifest
Conflict Detection	Name collision + false axiom	Redeclaration + false axiom

Table 1: Comparison of formulation approaches. Both detect incompatibility via Lean's module system; they differ in how model membership is expressed.

```
bash _lean/check_compatibility.sh
```

Output sections:

1. **EXAMPLE 1**: Structure-based formulation (ClassicalMechanics + FakeScience)
2. **EXAMPLE 2**: Predicate-based formulation (ClassicalMechanicsPredicate + FakeSciencePredicate)
3. **Summary**: Both examples verified for expected conflicts

This unified verification demonstrates that the two complementary approaches to formalizing Classical Mechanics both provide automatic, machine-checkable incompatibility detection when conflicting axiomatizations are introduced.

References

[1] J. C. C. McKinsey, A. C. Sugar, and Patrick Suppes. Axiomatic foundations of classical particle mechanics. *Journal of Rational Mechanics and Analysis*, 2(2):253–272, 1953.